Understanding Your Project - Plain-Language Explainer

Read this once, slowly, before the demo. It explains **what you built, why each part exists, and how the parts connect**, in the words you can use when she asks questions.

1. The project in one paragraph

You built a small web application, the **Device Compliance Management Portal**, where an organisation registers its computers, checks each one against security rules, and moves problems through a controlled workflow until they are fixed. Then - and this is the real subject of the course - you built the **automated delivery system around it**: every change is stored in Git, built and tested by Jenkins, packaged in Docker, and installed on a server by Ansible, with automatic rollback if a release is bad. **The app is the cargo; the DevOps pipeline is the delivery truck.** Most of the marks are for the truck.

2. Part A - the application (what it does)

- **Register a device**: name, owner, serial number, operating system, and three yes/no controls: antivirus, disk encryption, firewall.
- **Compliance rule** (one line of code): a device is **COMPLIANT only if all three controls are on**; otherwise **NON_COMPLIANT**. (The OS-version check in the original plan was left out.)
- **Search** by name or owner, filter by status; **edit** a device; **dashboard** with totals.
- **Three roles** with a login (passwords are stored hashed with BCrypt):

Role	What they may do in the workflow
Technician	Move a problem forward: Reported -> Under review -> Remediated
Auditor	Close a remediated device, or reopen it to Under review
Admin	Everything above

- **Workflow statuses**: REPORTED -> UNDER_REVIEW -> REMEDIATED -> CLOSED. Every move is written to a **history table** (who, when, from, to). The server refuses moves the role is not allowed to make.
- **How it is built**: Java 11 and **Spring Boot**. A browser request goes to a *controller* (handles the web request) -> a *service* (the rules: compliance and workflow) -> a *repository* (talks to the database) -> **H2 database** (a small file database). Pages are made with **Thymeleaf** templates. `/actuator/health` returns `{"status":"UP"}` - a standard "is it alive?" check that the pipeline and Ansible use.
- It is packaged as a single **WAR file** (one deployable file) that also runs on its own.

3. Part B - the delivery system (the DevOps part)

Think of an **assembly line**: raw change in, tested and installed product out, with quality gates that stop bad products.

Tool	The problem it solves	What you did with it
Git + GitHub	Track every change; work in parallel without overwriting each other	Branches per feature, pull requests with review comments, a deliberate merge conflict and its fix, tags `v1.0.0`, `v1.1.0`
Agile / Kanban	Plan the work and know when it is "done"	10 user stories, 15-task backlog, 3 sprints, Definition of Done
Maven	Build the app the same way every time	Compiles, runs the 11 unit tests, produces the WAR

Tool	The problem it solves	What you did with it
Jenkins	Do the build/test/deploy automatically on every change	Job `dcmp-ci` (builds on each change) and `dcmp-pipeline` (full pipeline)
Jenkinsfile	Keep the pipeline definition *in the repository* ("pipeline as code")	9 stages, listed below
Tomcat	A server that runs a WAR	Deployment target 1, port 8081; the environment (dev/qa) is a parameter
Selenium	Test the real web pages like a user would, in a real browser	5 user journeys (6 tests) that click through login, register, search, workflow, dashboard
Docker	"Works on my machine" -> runs identically everywhere	Dockerfile builds an image; Jenkins runs it as a container
Registry	A shelf for versioned images	Local registry; images tagged `dcmp:1.0.<build number>`
Ansible	Configure servers by a script instead of by hand	Playbook turns a blank Ubuntu server into a working portal server

The 9 pipeline stages: Checkout -> Build (+11 unit tests) -> Package -> **Selenium Tests** -> Deploy to Tomcat -> Verify (health check) -> Docker Build -> Docker Push -> Docker Deploy. **If any stage fails, everything after it is skipped** - that is the quality gate.

4. The journey of one change (explain it like this)

1. You change code and **commit** it; it goes to GitHub through a pull request.
2. **Jenkins** notices the new commit (it checks GitHub every 2 minutes) and starts the pipeline.
3. It **builds** the app and runs the **11 unit tests**.
4. It starts the new build on a private port (8082) with an empty database and runs the **6 Selenium browser tests**. *Fail here = the broken build goes nowhere.*
5. It **deploys** to Tomcat and checks `/actuator/health`.
6. It builds a **Docker image** `dcmp:1.0.N`, **pushes** it to the registry, and starts a **fresh container** from it.
7. Separately, **Ansible** can install a chosen release on a server, check its health, and **roll back** by itself if the check fails.

5. The two stories that prove it works (memorise these)

- **The deliberate defect (step 10).** You changed the compliance rule from AND to OR. All 11 unit tests still passed - they never tested that rule - but the Selenium test "device with only antivirus must be NON_COMPLIANT" failed, so Jenkins stopped before deploying (build #3: red at Selenium, Deploy skipped). You fixed it (commit `a1cfb33`) and build #4 went green. **Lesson: unit tests alone are not enough.**
- **The bad release (step 14).** Release 1.0.8 is a corrupted file. Ansible deployed it, the health check failed, and it **rolled back to 1.0.7 by itself** (the "current" link was pointed back and the service restarted). **Lesson: a failed deployment does not take the service down.**

6. Ideas you should be able to explain

- **CI vs CD**: *Continuous Integration* = every change is built and tested automatically. *Continuous Deployment* = a change that passes is released automatically.
- **Idempotent**: run the Ansible script again and nothing changes (`changed=0`) - safe to repeat.
- **Image vs container**: an image is the frozen recipe/package; a container is a running copy of it.

- **Rollback**: each release lives in its own folder; "current" is a pointer. Rolling back = move the pointer and restart.
- **Health check**: a URL the system calls to ask the app "are you OK?" - used by the pipeline, Docker and Ansible.
- **Port map**: 8080 = app default; 8081 Tomcat; 8082 Selenium test copy; 8092 Docker container; 8093 Ansible server; 9090 Jenkins; 5000 registry.

7. Honest limits (say them before she finds them)

- Demo accounts and an H2 file database - fine for a coursework demo, **not production**. There is no HTTPS and no CSRF protection (login is simple session-based, not Spring Security).
- The OS-version compliance rule and custom error pages were planned but not done.
- The Ansible target is a **container**, not a real server (so it uses supervisor instead of systemd). The "bad release" is a corrupted file, not a real faulty build.
- Rollback restores the program, **not the database**.
- Jenkins, Docker and Ansible all run on your own laptop; nothing is in the cloud.
- The Kanban board is documented in the repository (tables and diagrams), not a GitHub Projects board.
- Be straightforward about how you built it, including any AI assistance. What matters is that you can explain and run every part - this document and the demo guide exist for exactly that.

8. Quick facts card

Item	Value
Repository	github.com/Athdam05/device-compliance-portal
Tags	`v1.0.0` (MVP), `v1.1.0` (complete)
MVP tasks	15 (all done)
Tests	11 unit + 6 Selenium (5 journeys)
Pipeline	9 stages, ~2 min 20 s end to end
Failed / fixed builds	#3 (defect) / #4 (fix)
Auto-triggered build	#7 (started by a Git commit)
Roles	Admin, Auditor, Technician
Ansible rollback proof	release 1.0.8 rejected -> back on 1.0.7
Where everything is	the evidence report (PDF) and `docs/` in the repository